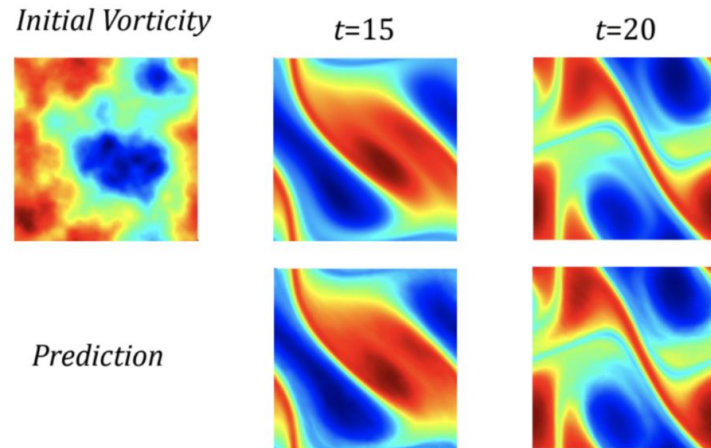


Fourier Neural Operator for Parametric Partial Differential Equations

<https://doi.org/10.48550/arXiv.2010.08895>



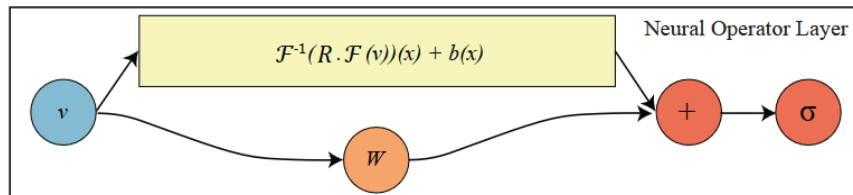
Authors

Zongyi Li, Nikola Kovachki, Burigede Liu, Kaushik
Bhattacharya, Andrew Stuart, Anima
Anandkumar - *California Institute of Technology*

Kamyar Azizzadenesheli - *Purdue University*

Presentation

Eric Zander - *University of Oregon*



Motivations

Partial Differential Equations (PDEs)

- PDEs: equations that describe relationships of quantities in terms of their rates of change (ie partial derivatives)
 - Often relate variation across space and time
- Notable quantities
 - Temperature
 - Pressure
 - Velocity
 - Density
 - Displacement
 - Electric Field
 - Magnetic Field
 - Heat Transfer
 - Fluid Flow
 - Chemical Diffusion
 - Structural Deformation
 - Strain
 - Probability Density
 - Time

$$\frac{\delta u}{\delta t} = f \left(u, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}, \dots \right)$$

General form of PDE relating time and space

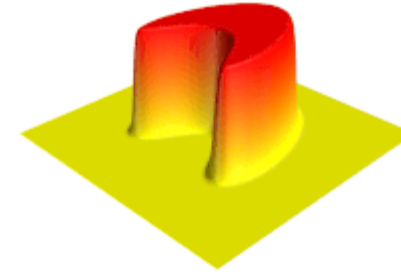
$$\frac{\partial u}{\partial t} = \nabla^2 u$$

Heat Equation (compact)

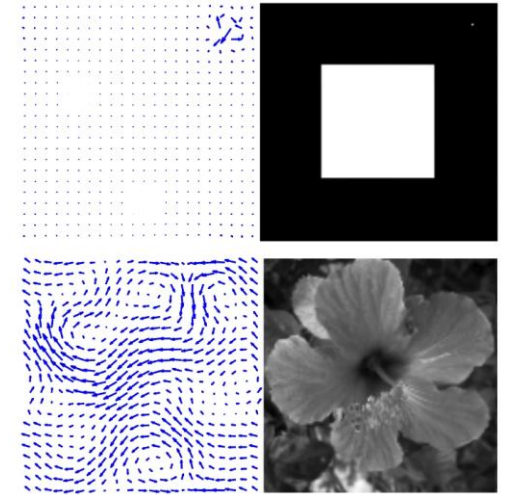
∇ = Laplacian $\approx$ sum of second spatial derivatives

Applications

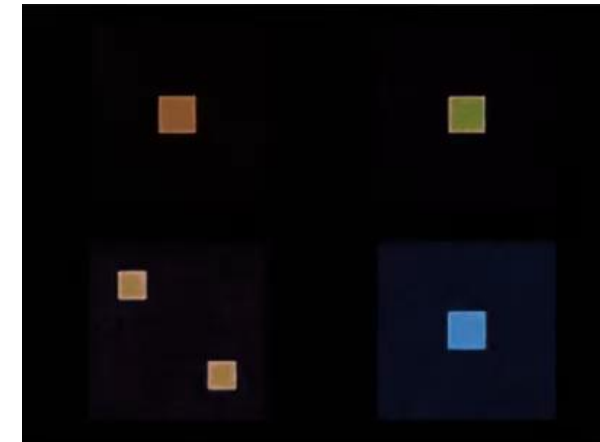
- Fluid simulation & aerodynamics
- Medical imaging & bioengineering
- Robotics & control systems
- Materials & structural analysis
- Basically all over in physics and engineering
 - PDEs are a foundational component of scientific computing



2D Heat Equation



Navier-Stokes Fluid Simulation

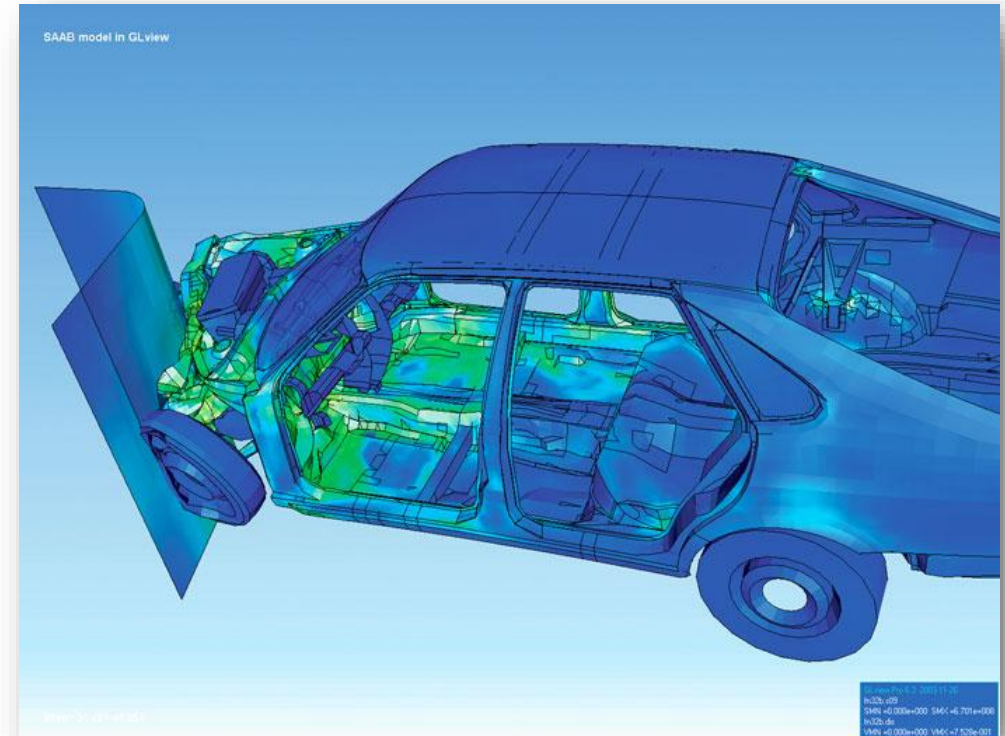


Turing patterns from Gray-Scott
model of reaction diffusion (Jens Luebeck)

Existing Methods

Conventional Solvers

- Discretize space and use numerical analysis methods
 - Finite element methods (FEM)
 - Finite difference methods (FDM)
- Tradeoff
 - Coarser grids are faster but have less resolution
 - Finer grids have better resolution but take longer
- Can be slow and inefficient



“Visualization of how a car deforms in an asymmetrical crash using finite element analysis”

https://commons.wikimedia.org/wiki/File:FAE_visualization.jpg



Data-Driven Methods

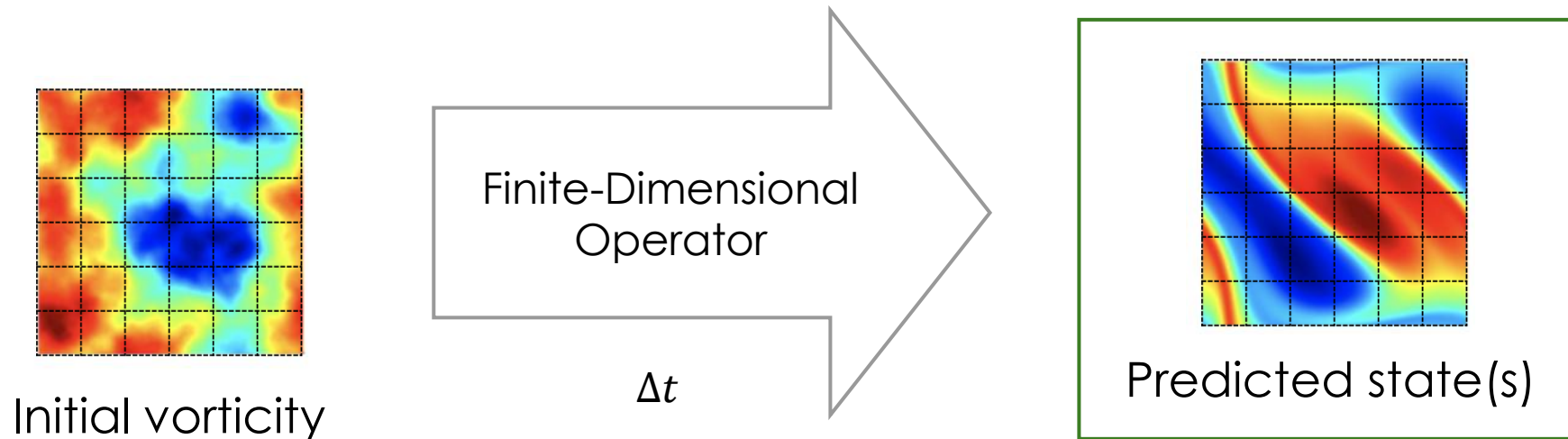
- Can potentially approximate solutions faster than conventional solvers
- Neural-FEM
 - Uses a network to represent a PDE solution (ie learns the solution)
 - Only works for one instance of a PDE
 - Must train a new model per instance
- Finite-Dimensional Operators
 - Converts function to fixed-size representation and learns mapping across a dimension like time (ie learns the operator)
 - Depends on discretization

Limitations

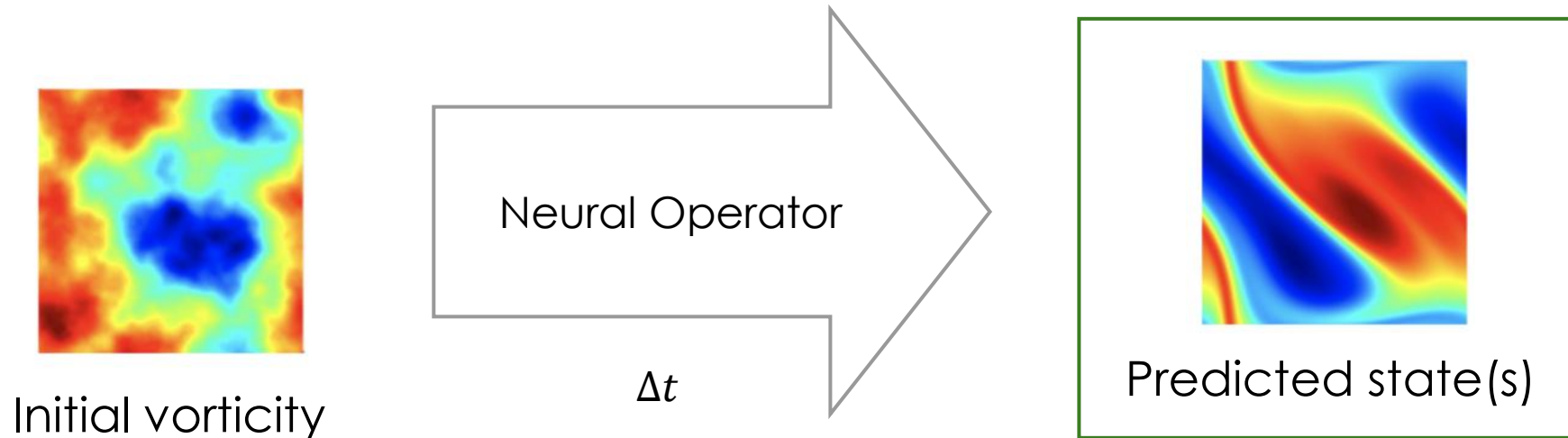
- Conventional methods
 - Can be slow and inefficient
- Neural-FEM (and PINNs)
 - Must train for each instance of PDE
- Finite-dimensional operators
 - Discretization dependence

Neural Operators and FNO

Neural Operators



Neural Operators



Neural Operators

Unlike networks that map between finite Euclidean spaces or sets, neural operators approximate mappings between infinite-dimensional function spaces (and Banach spaces in particular)

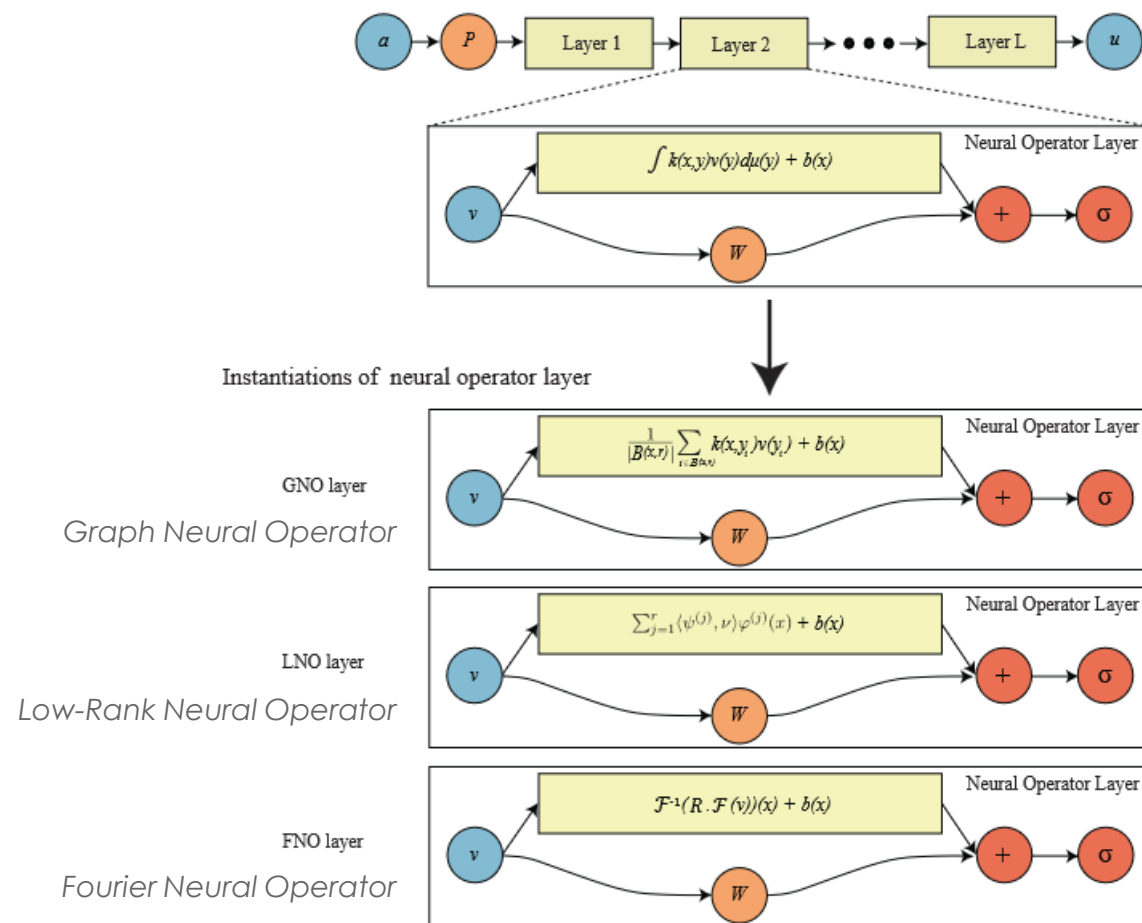


Figure 2: Neural operator architecture schematic

The input function a is passed to a pointwise lifting operator P that is followed by T layers of integral operators and pointwise non-linearity operations σ . In the end, the pointwise projection operator Q outputs the function u . Three instantiation of neural operator layers, GNO, LNO, and FNO are provided.

Neural Operators

Unlike networks that map between finite Euclidean spaces or sets, neural operators approximate mappings between infinite-dimensional function spaces (and Banach spaces in particular)

Banach spaces: Normed, complete vector spaces

i.e. Function spaces where distances are defined and convergence plays nice for optimization problems

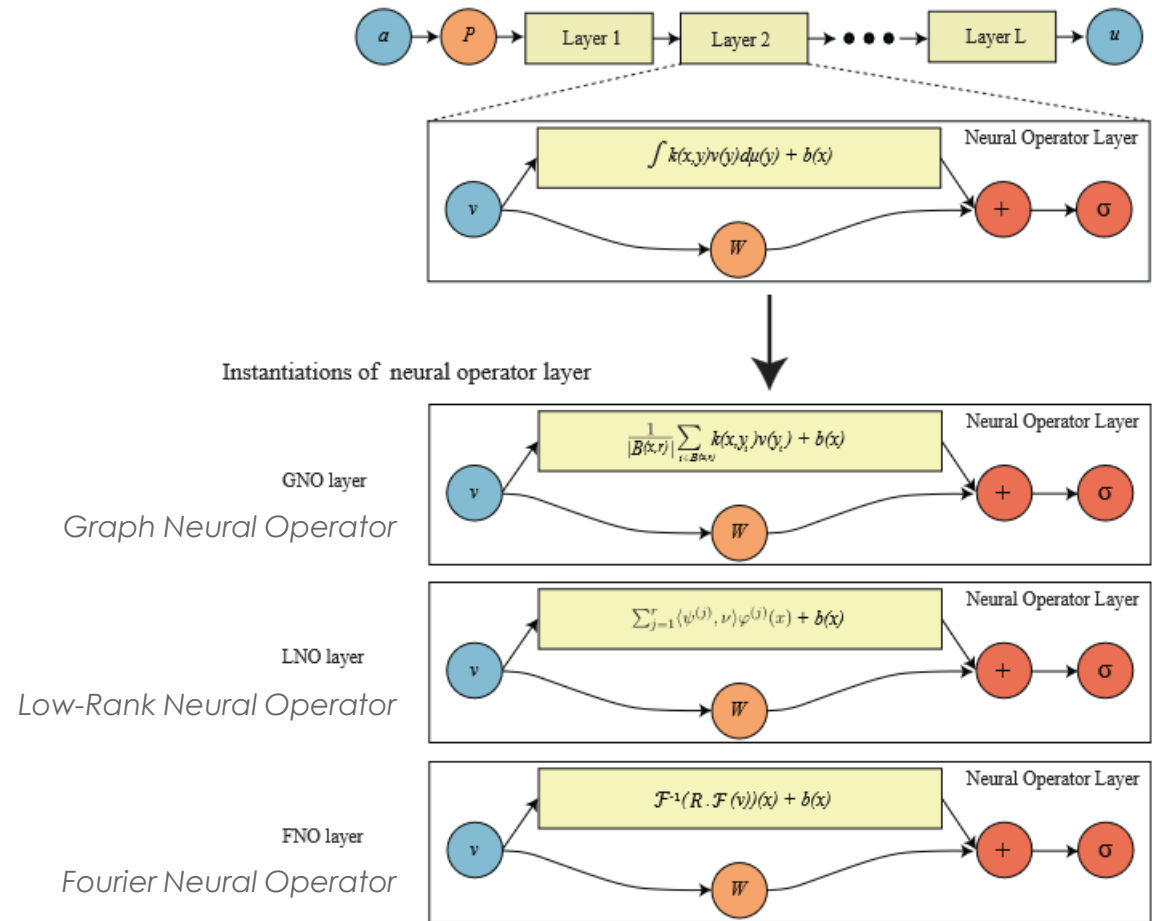


Figure 2: Neural operator architecture schematic

The input function a is passed to a pointwise lifting operator P that is followed by T layers of integral operators and pointwise non-linearity operations σ . In the end, the pointwise projection operator Q outputs the function u . Three instantiation of neural operator layers, GNO, LNO, and FNO are provided.

Kovachki et al., 2021

Neural Operators

Unlike networks that map between finite Euclidean spaces or sets, neural operators approximate mappings between infinite-dimensional function spaces (and Banach spaces in particular)

- Neural operators more naturally support problems emphasizing function spaces
 - e.g. Differential equations

Banach spaces: Normed, complete vector spaces

i.e. Function spaces where distances are defined and convergence plays nice for optimization problems

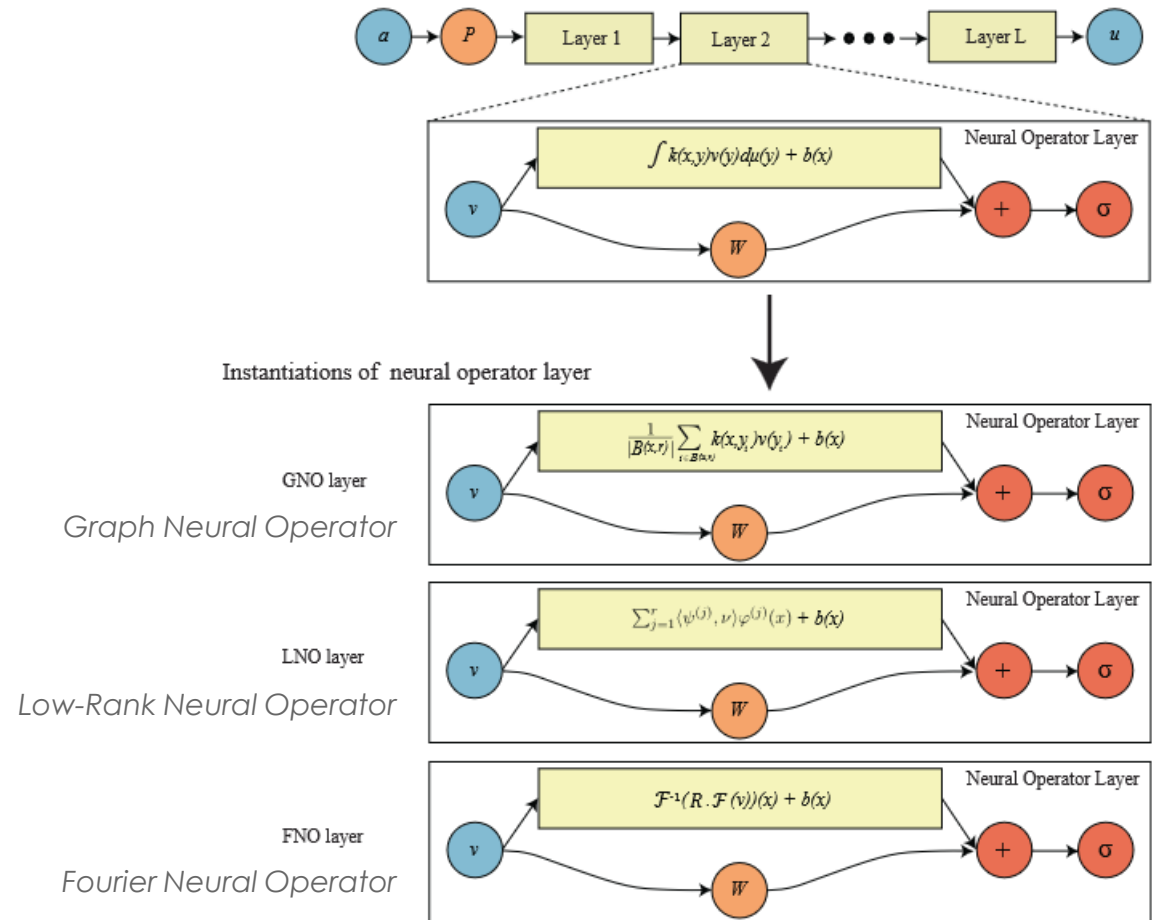


Figure 2: Neural operator architecture schematic

The input function a is passed to a pointwise lifting operator P that is followed by T layers of integral operators and pointwise non-linearity operations σ . In the end, the pointwise projection operator Q outputs the function u . Three instantiation of neural operator layers, GNO, LNO, and FNO are provided.

Kovachki et al., 2021

Neural Operators

Unlike networks that map between finite Euclidean spaces or sets, neural operators approximate mappings between infinite-dimensional function spaces (and Banach spaces in particular)

- Neural operators more naturally support problems emphasizing function spaces
 - e.g. Differential equations
- **They support discretization-invariance**
 - e.g. Train on one discretization such as 32 points and apply on another such as 128 points. Not perfectly though, just structurally possible!

Banach spaces: Normed, complete vector spaces

i.e. Function spaces where distances are defined and convergence plays nice for optimization problems

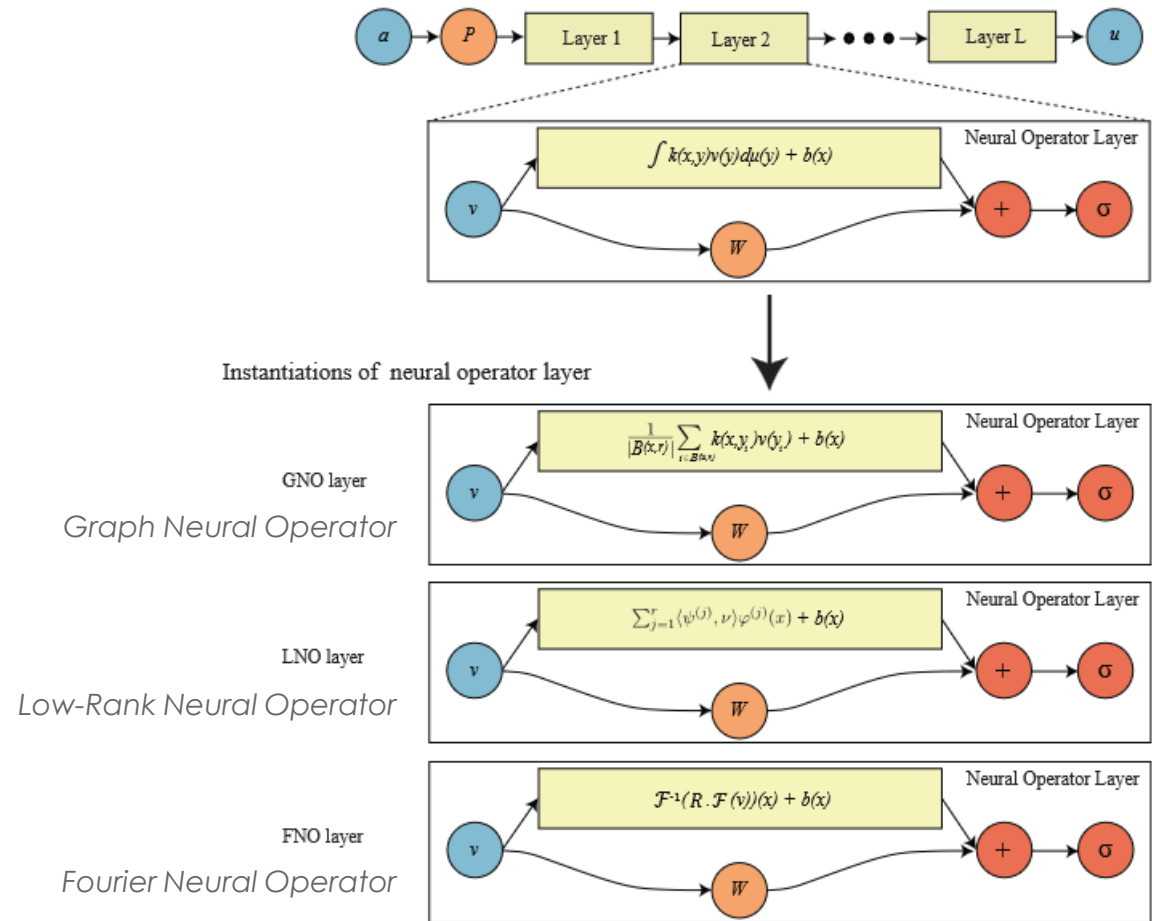


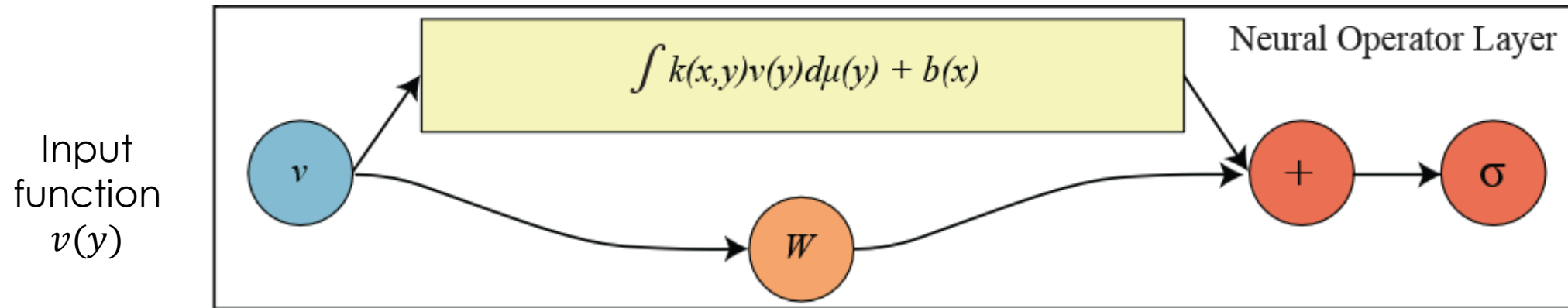
Figure 2: Neural operator architecture schematic

The input function a is passed to a pointwise lifting operator P that is followed by T layers of integral operators and pointwise non-linearity operations σ . In the end, the pointwise projection operator Q outputs the function u . Three instantiation of neural operator layers, GNO, LNO, and FNO are provided.

Kovachki et al., 2021

Neural Operators

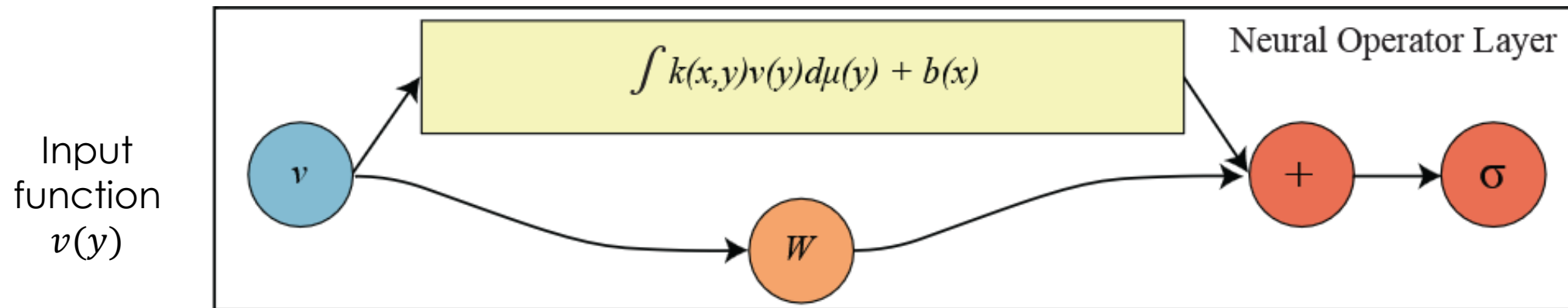
The idealized operation is a weighted (k) aggregation of all points (y) for a point (x) with a local bias (b). Like a continuous, non-local generalization of a convolution.



Neural Operators

The idealized operation is a weighted (k) aggregation of all points (y) for a point (x) with a local bias (b). Like a continuous, non-local generalization of a convolution.

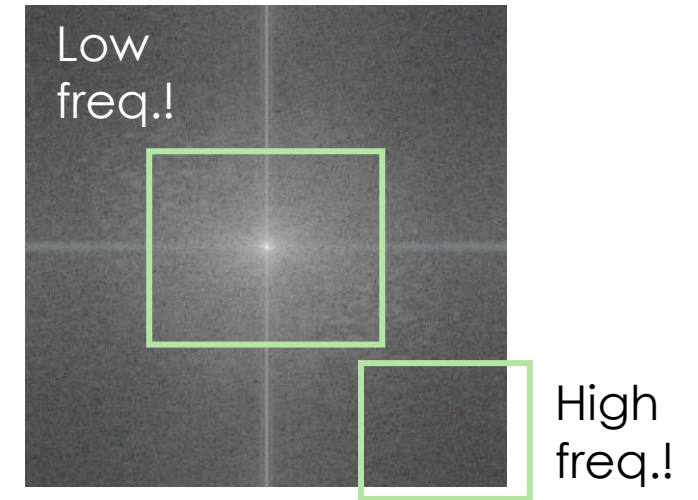
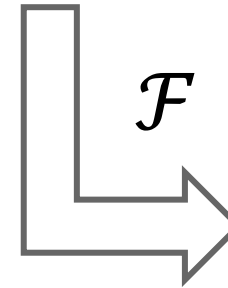
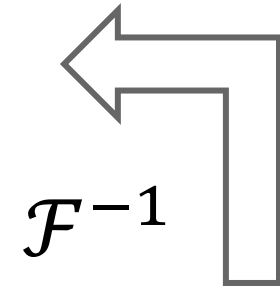
However, in practice this is too damn expensive ($O(N^2)$ when discretized) and will explode your prized Dell Inspiron 4100



Frequency Domain



Spatial domain



Frequency domain

Frequency Domain

To practically achieve the core neural operator, we can use an FFT F , retain only valuable low frequency modes I for efficiency, perform a convolution by multiplying in the frequency domain, and then use an inverse FFT F^{-1} .

This convolution is spatially non-local and works due to the principles described by the convolution theorem.

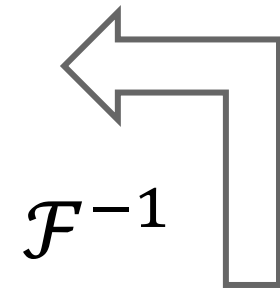
Convolution Theorem

$$f(x, y) * h(x, y) \Leftrightarrow F(u, v)H(u, v)$$

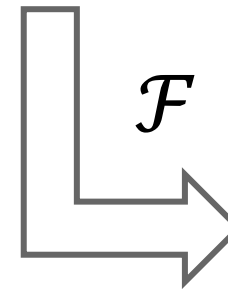
Convolution in the spatial domain is equivalent to multiplication in the frequency domain



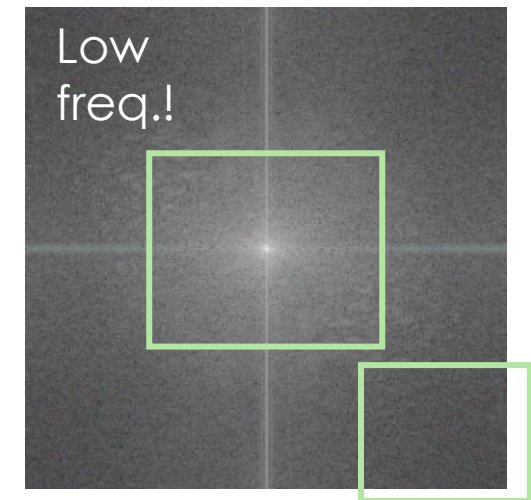
Spatial domain



F^{-1}



F



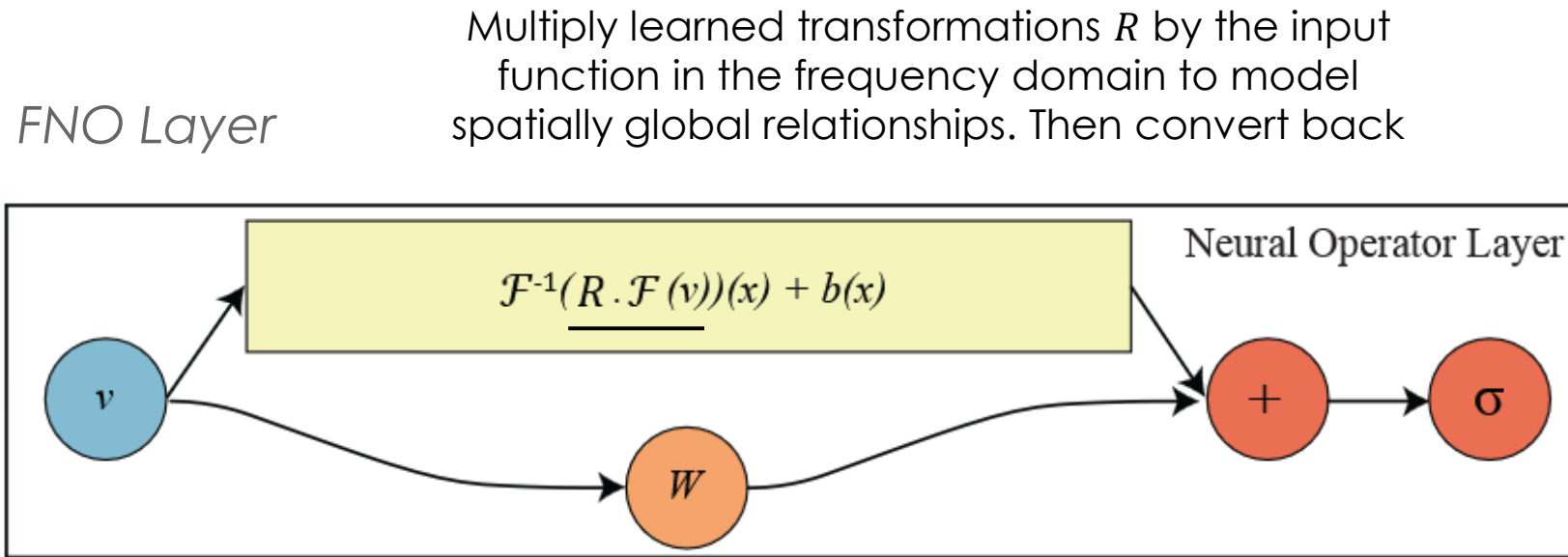
Low
freq.!

High
freq.!

Frequency domain



Fourier Neural Operators (FNO)

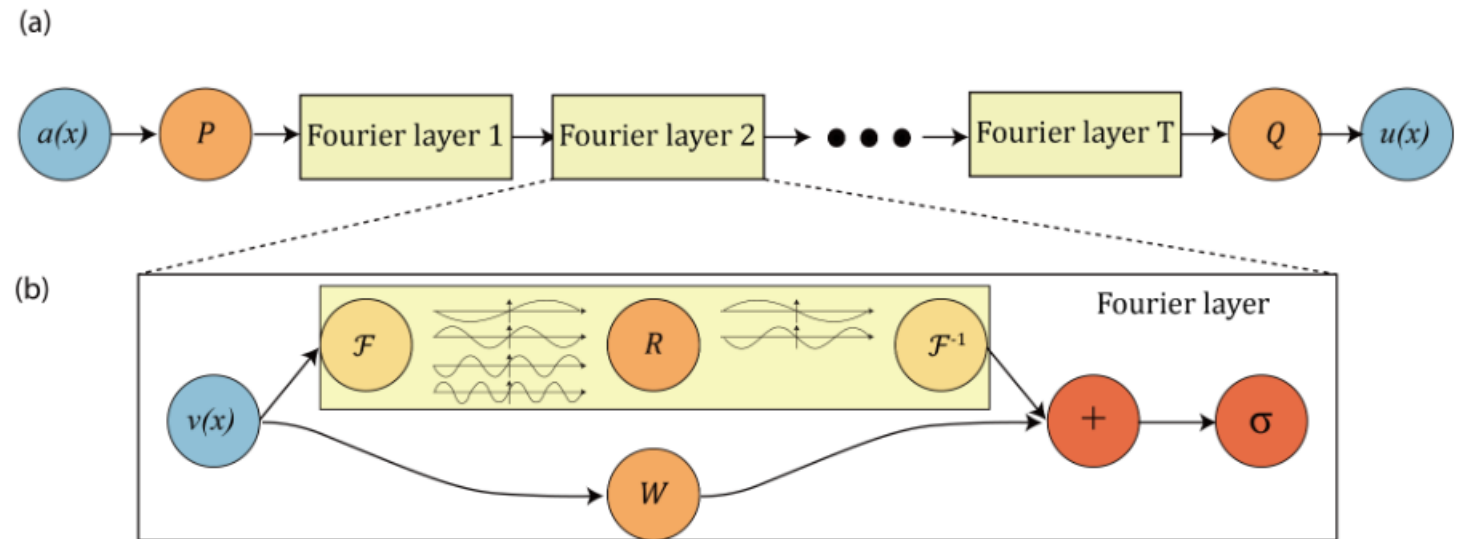


W is a local linear transform to retain spatially local pointwise transformations and complement the operation in the frequency domain

Fourier Neural Operators (FNO)

The authors argue that most of the cost lies in computing the FFT and its inverse.

This makes an FNO layer $O(n \log n)$

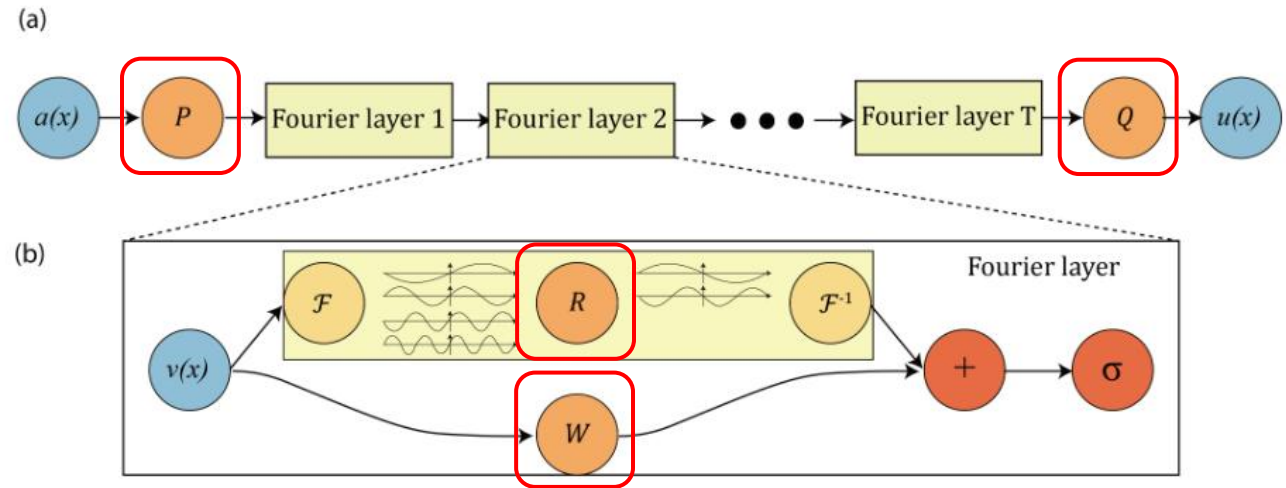


(a) **The full architecture of neural operator:** start from input a . 1. Lift to a higher dimension channel space by a neural network P . 2. Apply four layers of integral operators and activation functions. 3. Project back to the target dimension by a neural network Q . Output u . (b) **Fourier layers:** Start from input v . On top: apply the Fourier transform $\mathcal{F}$; a linear transform R on the lower Fourier modes and filters out the higher modes; then apply the inverse Fourier transform $\mathcal{F}^{-1}$. On the bottom: apply a local linear transform W .

Figure 2: **top:** The architecture of the neural operators; **bottom:** Fourier layer.

Training

- F and F^{-1} are fixed
- Need to train:
 - Lifting operator P
 - Fourier layer weights R
 - Local linear transformation W
 - Projection operator Q
- Fairly standard loss and backpropagation



Numerical Experiments

Burgers' Equation

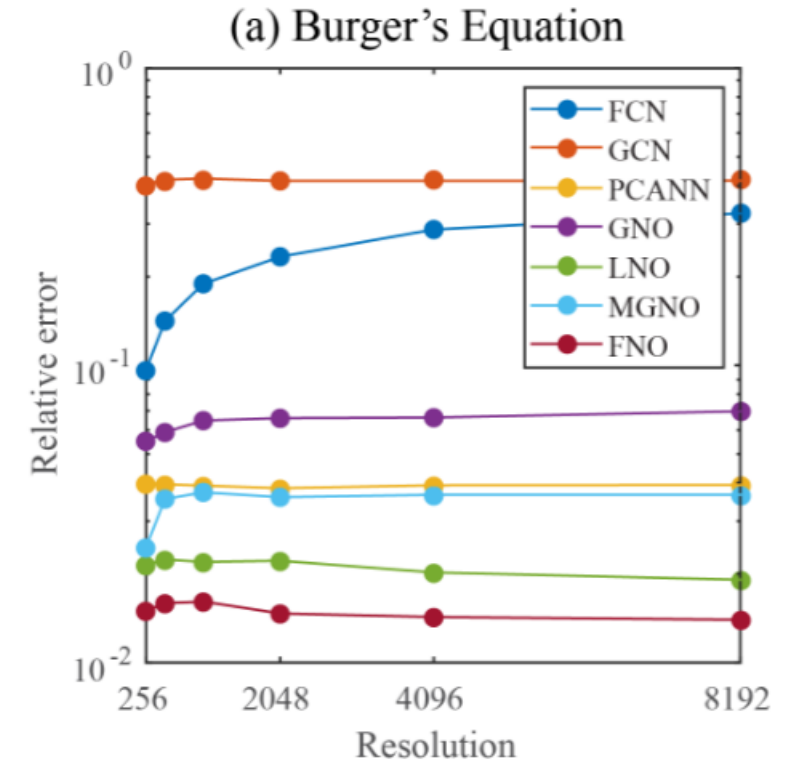
$$\begin{aligned}\partial_t u(x, t) + \partial_x(u^2(x, t)/2) &= \nu \partial_{xx} u(x, t), & x \in (0, 1), t \in (0, 1] \\ u(x, 0) &= u_0(x), & x \in (0, 1)\end{aligned}$$

- Non-linear, 1D Burgers' equation
- For modelling viscous fluid

Table 3: Benchmarks on 1-d Burgers' equation

Networks	$s = 256$	$s = 512$	$s = 1024$	$s = 2048$	$s = 4096$	$s = 8192$
NN	0.4714	0.4561	0.4803	0.4645	0.4779	0.4452
GCN	0.3999	0.4138	0.4176	0.4157	0.4191	0.4198
FCN	0.0958	0.1407	0.1877	0.2313	0.2855	0.3238
PCANN	0.0398	0.0395	0.0391	0.0383	0.0392	0.0393
GNO	0.0555	0.0594	0.0651	0.0663	0.0666	0.0699
LNO	0.0212	0.0221	0.0217	0.0219	0.0200	0.0189
MGNO	0.0243	0.0355	0.0374	0.0360	0.0364	0.0364
FNO	0.0149	0.0158	0.0160	0.0146	0.0142	0.0139

s : Resolution of discretization



Darcy Flow

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x) \quad x \in (0, 1)^2$$

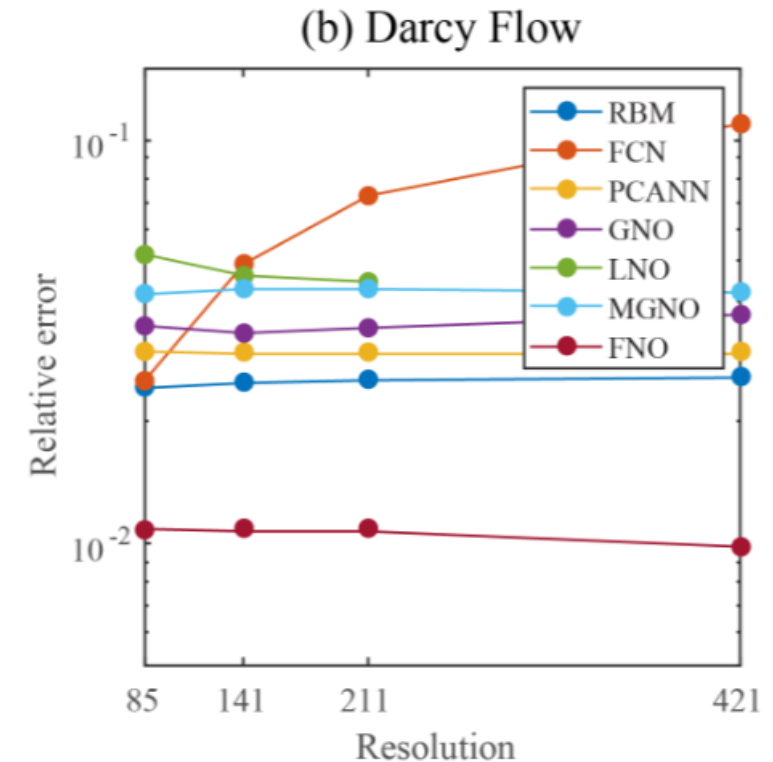
$$u(x) = 0 \quad x \in \partial(0, 1)^2$$

- Steady-state of the 2D Darcy Flow equation
- Applications in modeling flow, material deformation, and electric potential

Table 4: Benchmarks on 2-d Darcy Flow

Networks	$s = 85$	$s = 141$	$s = 211$	$s = 421$
NN	0.1716	0.1716	0.1716	0.1716
FCN	0.0253	0.0493	0.0727	0.1097
PCANN	0.0299	0.0298	0.0298	0.0299
RBM	0.0244	0.0251	0.0255	0.0259
GNO	0.0346	0.0332	0.0342	0.0369
LNO	0.0520	0.0461	0.0445	—
MGNO	0.0416	0.0428	0.0428	0.0420
FNO	0.0108	0.0109	0.0109	0.0098

s : Resolution of discretization



They don't clearly state error, I think it's L^2

Navier-Stokes Equation

- Applications in modelling viscous flow
- 2D and 3D configurations
 - 2D: CNN + RNN 
 - 3D: CNN with full trajectory as 3D 

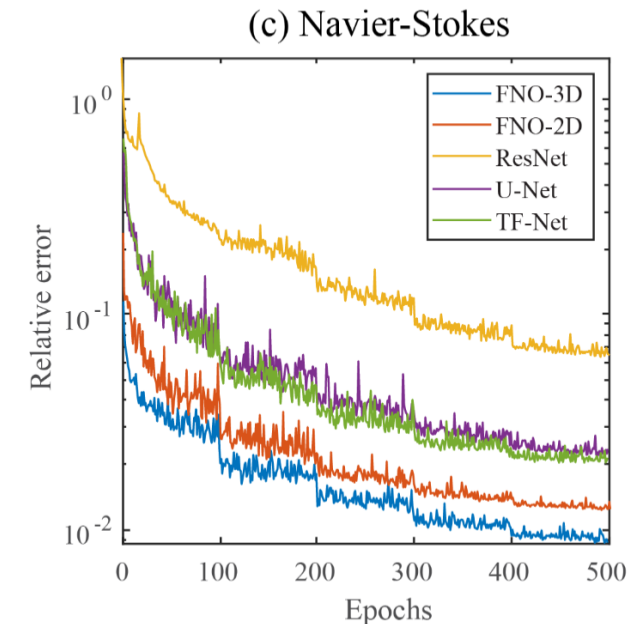


Navier-Stokes can be used to model viscous flow

$$\begin{aligned} \partial_t w(x, t) + u(x, t) \cdot \nabla w(x, t) &= \nu \Delta w(x, t) + f(x), & x \in (0, 1)^2, t \in (0, T] \\ \nabla \cdot u(x, t) &= 0, & x \in (0, 1)^2, t \in [0, T] \\ w(x, 0) &= w_0(x), & x \in (0, 1)^2 \end{aligned}$$

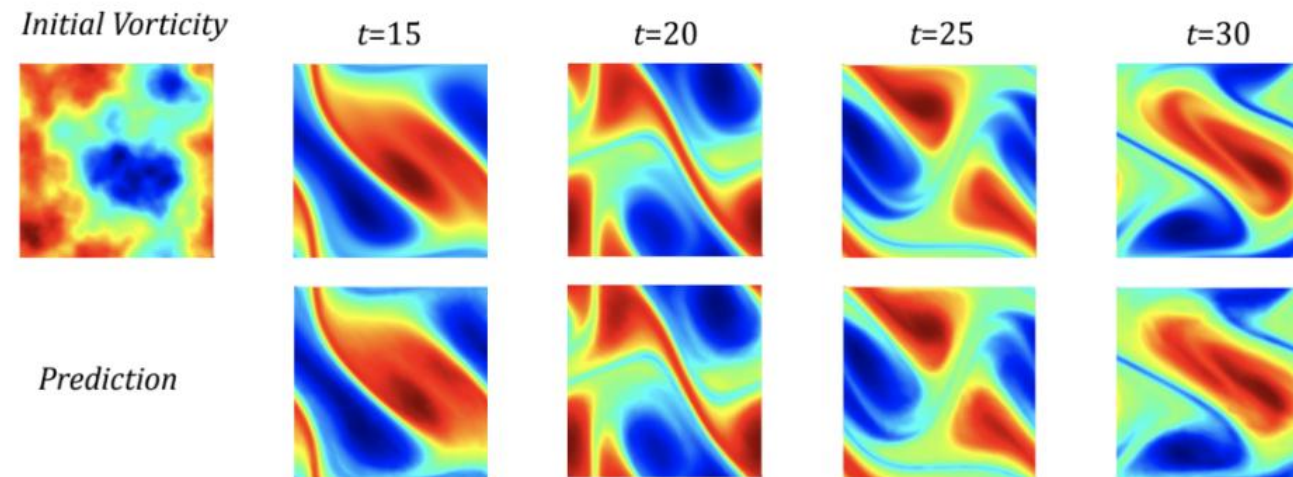
Config	Parameters	Time per epoch	$\nu = 1e-3$ $T = 50$ $N = 1000$	$\nu = 1e-4$ $T = 30$ $N = 1000$	$\nu = 1e-4$ $T = 30$ $N = 10000$	$\nu = 1e-5$ $T = 20$ $N = 1000$
FNO-3D	6, 558, 537	38.99s	0.0086	0.1918	0.0820	0.1893
FNO-2D	414, 517	127.80s	0.0128	0.1559	0.0834	0.1556
U-Net	24, 950, 491	48.67s	0.0245	0.2051	0.1190	0.1982
TF-Net	7, 451, 724	47.21s	0.0225	0.2253	0.1168	0.2268
ResNet	266, 641	78.47s	0.0701	0.2871	0.2311	0.2753

For a spatial resolution of 64x64



Navier-Stokes Equation: Super Resolution

- To show handling of different resolutions
 - Training on $64 \times 64 \times 20$, inference on $256 \times 256 \times 80$
- Authors claim FNO-2D is the only model to be able to do zero-shot super-resolution



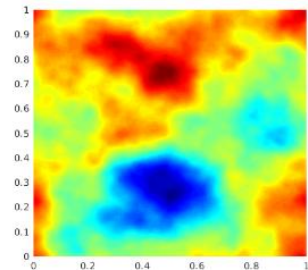
Zero-shot super-resolution: Navier-Stokes Equation with viscosity $\nu = 1e-4$; Ground truth on top and prediction on bottom; trained on $64 \times 64 \times 20$ dataset; evaluated on $256 \times 256 \times 80$ (see Section 5.4).

Figure 1: **top:** The architecture of the Fourier layer; **bottom:** Example flow from Navier-Stokes.

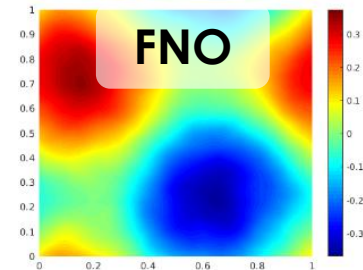
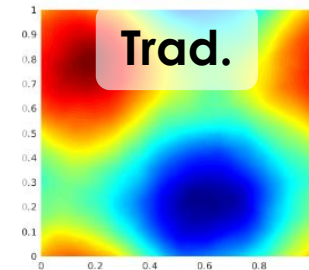
Navier-Stokes Equation: Bayesian Inverse Problem

- Used MCMC to draw samples for Navier Stokes at $T=50$
- Problem: Find initial vorticity by going backwards
- MCMC + Traditional solver: **18 hours**; MCMC + FNO: **2.5 minutes**

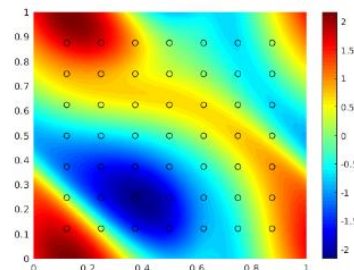
Real Initial
Vorticity



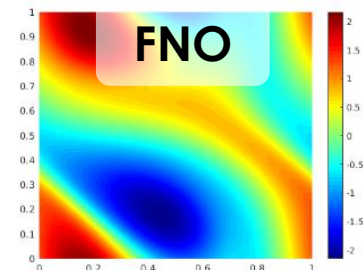
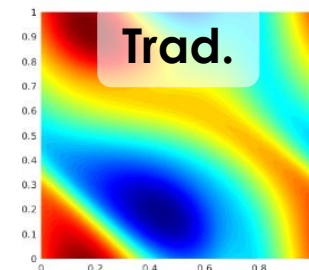
Estimated
initial
conditions



Observed
Vorticity at
 $T=50$ (MCMC)



$T=50$



Further Reading and Discussion

References and Further Reading

- Z. Li *et al.*, 'Fourier Neural Operator for Parametric Partial Differential Equations', *arXiv [cs.LG]*. 2021. <https://arxiv.org/abs/2010.08895>
- N. Kovachki *et al.*, 'Neural operator: learning maps between function spaces with applications to PDEs', *arXiv [cs.LG]*. 2021. <https://arxiv.org/abs/2108.08481>
- M. Xu *et al.*, 'Equivariant Graph Neural Operator for Modeling 3D Dynamics', *arXiv [cs.LG]*. 2024. <https://arxiv.org/abs/2401.11037>
- A. Bacho *et al.*, 'Operator Learning at Machine Precision', *arXiv [cs.LG]*. 2025. <https://arxiv.org/abs/2511.19980>

Discussion Questions

1. The authors highlight discretization-invariance as a benefit of FNO. Though inference for different resolutions is structurally possible, how practical do you think it is?
2. What are the tradeoffs between FNO and physics-informed approaches such as Neural-FEMs and PINNs?
3. The paper uses Fourier representations to capture global relationships in a system. What kind of problems might benefit from this, and where might it struggle?
4. If a traditional solver and a learned model produce similar results, would you have concerns about deploying the latter in a real application?

